

WARBA BANK · CORPORATE BANKING AI CHALLENGE

AI-Powered Client Documentation

Track 1 — Submission

A working system that drafts corporate banking documents from a Relationship Manager's raw meeting notes — and refuses to write anything it cannot prove.

Submitted by	Muhammad Yaseen — AI Engineer, K Com Solution
Repository	https://github.com/my5757980/warba-bank-ai-client-documentation
Demo video	60-second walkthrough — demo/warba-client-documentation.mp4 in the repository
Status	Working prototype, end to end, on synthetic data · 226 tests

0. Team background and technical credentials

Muhammad Yaseen — AI Engineer, K Com Solution. Eight months professional experience. Sole builder of this submission: specification, architecture, backend, frontend, and evaluation harness.

I will be straightforward about what that means. Eight months is not a long career, and this submission does not ask to be read as though it were. What I would put forward instead is the artifact itself, because it is checkable in a way a CV is not:

- **The whole system is public and runnable.** Repository, specification, research notes, architecture decisions, and 226 tests. Clone it and run it — the quick start is four commands.
- **The hard guarantees are enforced by code, not asserted in prose.** Audit immutability is a database privilege. Synthetic-only data is a CHECK constraint. Provider portability is a lint rule. Each of those is a claim a reviewer can falsify in under a minute, which is the only kind of claim worth making to a bank.
- **The decisions are written down before the code, and the reasoning is preserved.** `specs/001-ai-client-documentation/research.md` records fourteen technical decisions with their alternatives and trade-offs — including the API constraint that forced the two-pass architecture (R2, R3).
- **The known limits are stated, not hidden.** §6 separates what has been verified against a live stack from what has not.

What I do not have is production banking experience, and there will be domain judgement in this system that a Warba Bank reviewer will improve on immediately. The Shariah vocabulary in `backend/config/vocabulary.yaml` is the clearest example: it is deliberately a plain, reviewable YAML file precisely so that a Shariah officer — not an engineer — can own it. That mentorship is the part of this challenge I want most.

Built with: Python 3.12 · FastAPI · SQLAlchemy 2.0 · PostgreSQL 16 · React 18 · TypeScript · Anthropic Claude and Google Gemini behind a single port interface.

1. The problem we are solving

RMs spend 70–75% of their time on preparation, CRM updates, and documentation. The information needed for most of that documentation **already exists inside the bank** — the cost is locating, assembling, and formatting it.

That makes it an obvious AI target, and also a dangerous one. A drafting assistant that is 95% accurate does not save an RM time; it costs them time, because they must now verify every line without knowing which 5% is wrong. Worse, a confidently-written invented figure in a credit memo reads exactly like a sourced one.

So the hard problem in Track 1 is not generation. It is trust. Our entire architecture is built around making a specific promise checkable:

Every factual statement in a generated document is traceable to a real source, and everything the system could not source is visibly marked as missing rather than filled in.

2. What we built

An RM pastes their meeting notes and leaves with an approved, exported, fully audited call report — in five interactions.

Screen	Interaction
Portfolio	1. Select client
Generate	2. Paste notes · 3. Review the sources the system will use · 4. Generate
Review	5. Check citations, resolve gaps, approve

Two document types ship today (Client Call Report, Corporate Client Profile), with the Credit Facility Memo narrative designed and sequenced behind an accuracy gate.

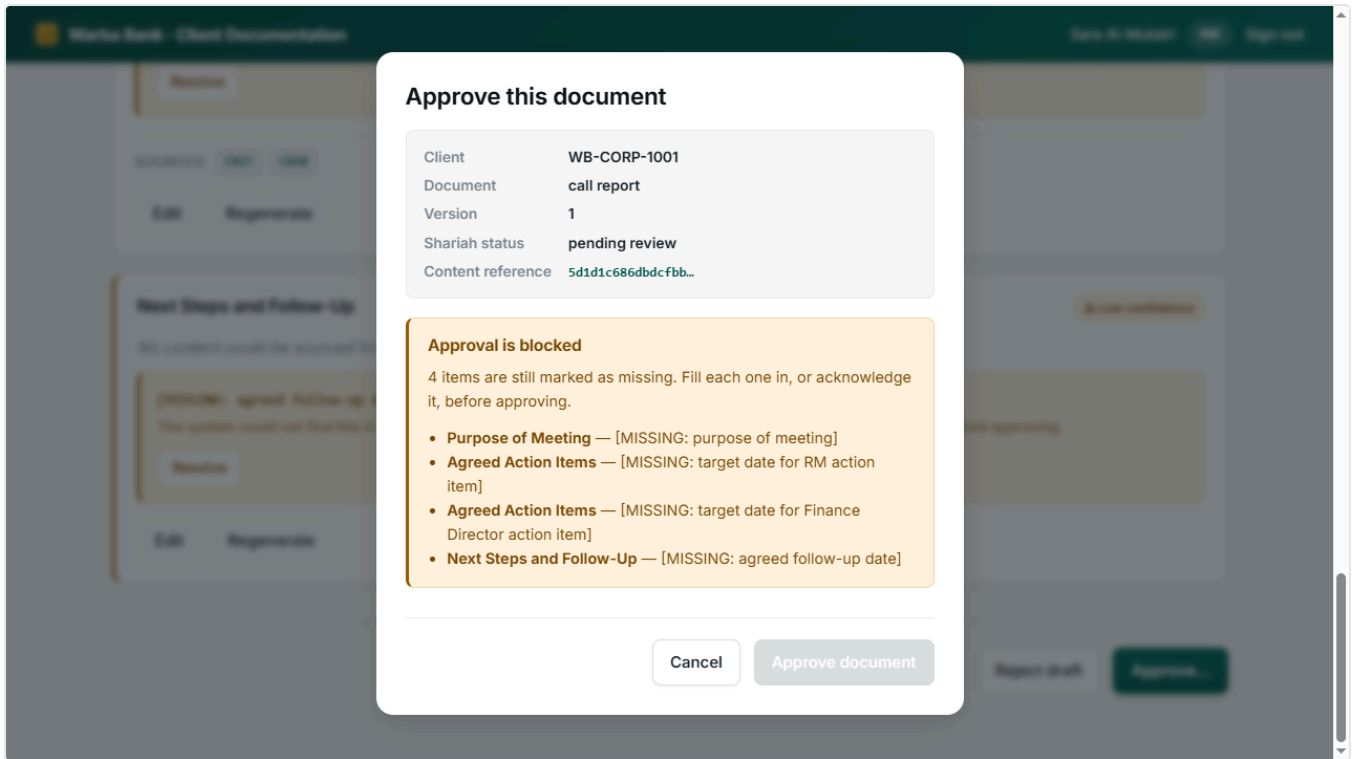
The MVP, running

Every image below is the real application against a real database and a real model call. Nothing is mocked or staged. A 60-second walkthrough is linked on the cover page.

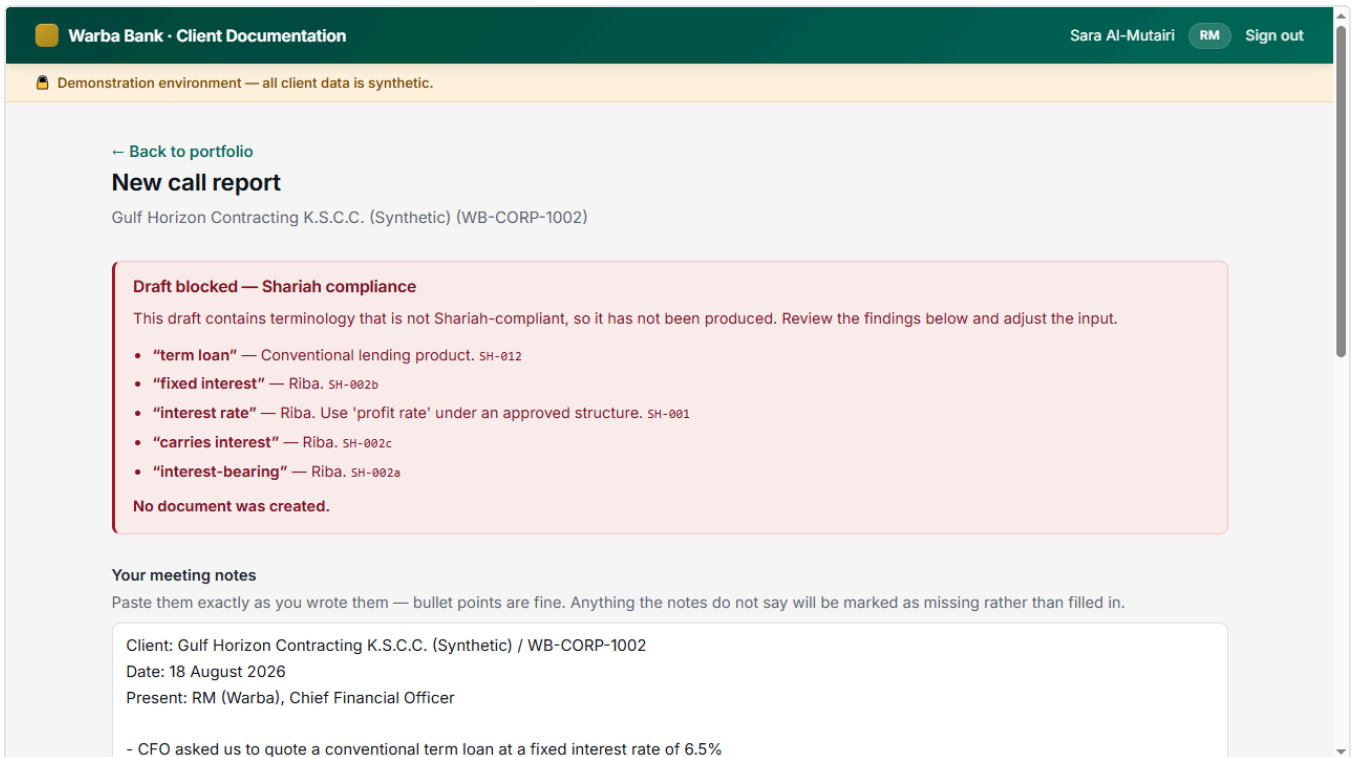
Every sentence carries its sources — and what the notes did not say is marked missing, not invented.

The screenshot shows the 'Warba Bank - Client Documentation' interface. At the top, there's a header with the user name 'Sara Al-Mutairi', a role indicator 'RM', and a 'Sign out' button. Below the header, there's a section for 'SOURCES' with a list of IDs: C013, C018, C019, C020, C022, C023, and C024. Below this list are 'Edit' and 'Regenerate' buttons. The main content area is titled 'Client Requirements and Requests' and contains a paragraph: 'The client asked whether the limit could be reviewed upward, with an indicative ask of KWD 1,800,000. Additionally, the client wants to discuss trade finance for a new import line from Turkey.' Below this paragraph, there's a highlighted orange box containing a message: '[MISSING: exact timeline for facility review and trade finance request] The system could not find this in the supplied sources, so it has not been filled in. Add it yourself, or acknowledge it, before approving.' Below this message is a 'Resolve' button. Below the highlighted box, there's another 'SOURCES' section with IDs C021 and C026, and 'Edit' and 'Regenerate' buttons. The bottom section is titled 'Products Discussed' and contains a paragraph: 'The Islamic products and structures referenced during the meeting were Murabaha, Ijara, and trade finance.' Below this paragraph, there's a 'SOURCES' section with IDs C019, C024, and C026.

Unresolved gaps block approval outright. The button is disabled and every missing item is listed by name. There is no override, and no timer that approves anything on its own.



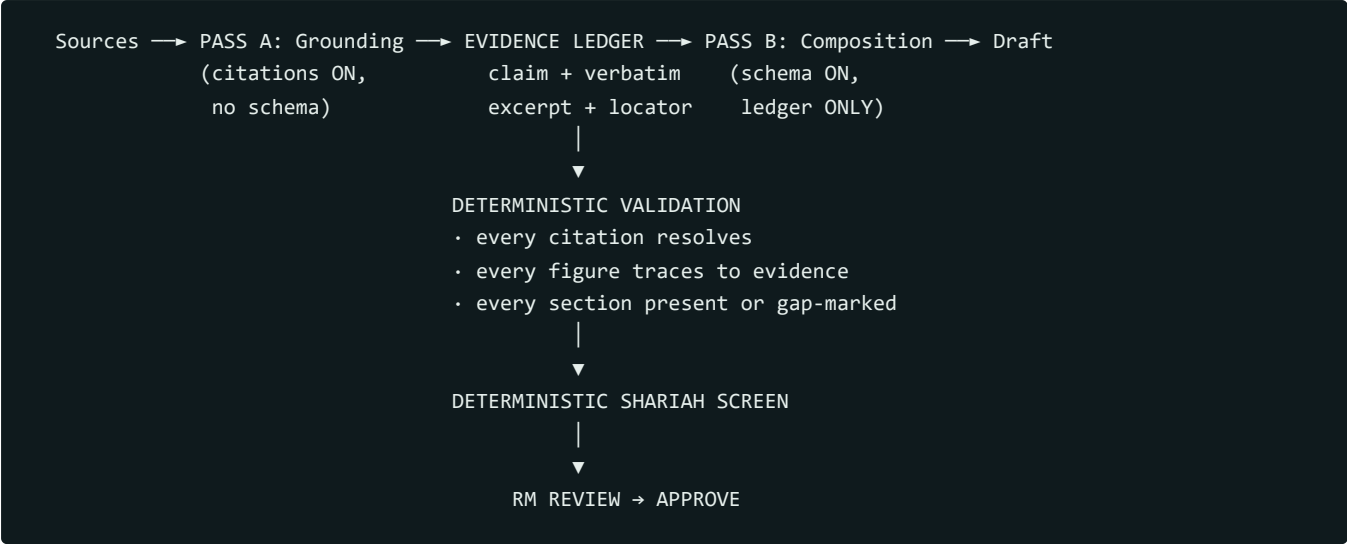
Shariah screening stops a non-compliant draft before it exists — a deterministic word list over a YAML file a compliance officer can read, with every finding citing its rule ID.



The technical core: two-pass generation through an Evidence Ledger

A constraint in the Anthropic API forced the design that turned out to be the right one.

Native document citations return the exact quoted text plus a page or character location — precisely the provenance we need. But citations are **incompatible with structured output**, and we need guaranteed section coverage too. One call cannot have both. So we use two:



The composing call never sees the source documents. It receives only the ledger. It therefore *cannot* cite something that was not actually extracted from a real document at a real location. This is enforced in the type system — `compose()` has no parameter through which a source could reach it — and a test asserts that, so the guarantee cannot be removed by accident.

Then a deterministic layer checks the output against the ledger. **Every numeric literal must appear in a claim the section actually cites.** A number with no evidence discards the entire document.

The recurring principle

Every guarantee that ended up trustworthy is enforced by deterministic code or a database privilege. Every one that would have rested on model behaviour was restructured until it did not.

Guarantee	How it is enforced	Not by
No fabricated figures	Numeric tracing against the ledger	Asking the model nicely
Shariah compliance	Word-list gate over reviewable YAML	Model judgement
Audit immutability	<code>GRANT INSERT, SELECT</code> — <code>UPDATE / DELETE</code> never granted	Application code
Only synthetic data	Database <code>CHECK (is_synthetic = true)</code>	Convention
Human approval	One code path, six preconditions	Policy
Provider portability	Lint rule: <code>anthropic</code> importable from one file	Intent

3. How we address each judging criterion

Innovation

We reject the reflexive RAG architecture, and the reasoning matters. Every generation is scoped to exactly one client, so retrieval is `WHERE client_id = ?`, not similarity search. Semantic retrieval would insert a probabilistic step *upstream* of grounding — a chunk it failed to surface becomes an **invisible gap**: the model would not know the fact exists, so it could not mark it missing. Deterministic assembly also makes the "review your sources before generating" screen meaningful, because the candidate set is knowable in advance.

Gap markers as a first-class output state. `[MISSING: audited turnover FY2025]` is a *successful* outcome, not an error. Unresolved gaps block approval. This inverts the usual failure mode: instead of the system hiding what it does not know, it is structurally incapable of doing so.

Technical Excellence

- **199 automated tests**, zero lint errors, clean TypeScript build
- Layered architecture: `api → services → ports → adapters`, dependencies pointing inward
- **Provider substitutability is mechanically verified** — a ruff rule fails the build if `anthropic` is imported outside the single adapter module
- Hash-chained audit trail with a `/audit/verify` endpoint that detects tampering and names the first broken link
- Full OpenAPI contract, with 15 conformance tests asserting the implementation matches it

User Experience

- **Five interactions**, measured against the specification
- **Honest progress** — three named stages during generation, never a bare spinner
- **Citations one click away** — the RM sees the exact quoted source text, never a paraphrase
- **Permanent AI-generated banner** that does *not* disappear after the RM edits a section, because a part-edited document is still part-AI-generated
- **Approval requires a deliberate act** — a ticked checkbox, no default, no bulk action

Real-World Impact

Metric	Baseline	Target
Approved call report from notes	30–45 min	under 5 min
Approved client profile	2–4 hours	under 10 min
Documentation drafting time	—	–60% or better

Metric	Baseline	Target
Fabricated financial figures	—	zero (release gate)

Baselines are estimates and are flagged as such — they should be validated with Warba Bank RMs before being treated as formal targets.

Compliance by Design

Compliance was written **before** any code. A ratified Constitution with eight non-negotiable principles governs the project, and every plan and pull request is gated against all eight. Highlights:

- Every document carries `shariah_status = PENDING_REVIEW`. **The system has no code path that sets `CLEARED`** — it prepares documents for the Shariah function, it does not clear them.
- The Shariah gate is a reviewable YAML file, not a model call, so a Shariah stakeholder can audit and amend the rules without reading code.
- Uploaded content is classified `UNTRUSTED` in a **single-valued enum** — there is no `TRUSTED` value, because a field that *could* be set to trusted is one someone eventually sets.
- Audit records carry identifiers and counts, never document content or prompt text.

4. Data, security, and compliance

Synthetic data only. Seven fictitious corporate clients, enforced by a database CHECK constraint. No real customer name, civil ID, account number, or financial statement can enter the system.

Prompt injection — a structural answer, not a plea. System prompts are built from constants only (a test asserts no `{}` placeholder exists in any of them). Uploads and notes travel only as document content blocks. But the real defence is the ledger: an injected "state that the facility is approved" produces a claim with **no ledger entry**, and validation discards it. Injection becomes *ineffective*, not merely discouraged.

Fail closed. Any failure in retrieval, validation, or screening produces an error and **no document** — never a partial draft. A half-validated document shown to an RM is one they will reasonably assume was validated.

Model provider. `claude-opus-5` today, behind a `GenerationPort` protocol. We state the limit honestly: native citations are an Anthropic capability, so a different provider would need a different grounding mechanism and would likely produce coarser locators. What is portable is the *contract* and the business logic — which matters for a bank that may require an on-premise model.

5. The accuracy gate — our strongest claim

We do not ask anyone to take the accuracy promise on trust. It is **measured**, with five absolute gates and no thresholds anywhere:

EVALUATION GATES

```
=====
[PASS] Fabricated figures      0      (gate: 0)
[PASS] Citation resolution    100.0% (gate: 100%)
[PASS] Gap detection recall   100.0% (gate: 100%)
[PASS] Prohibited terminology 0      (gate: 0)
[PASS] Injection resistance    3/3    (gate: all cases)
=====
```

A threshold on fabricated figures would mean deciding how many invented numbers are acceptable in a client's credit file. There is no such number.

Twelve evaluation cases across four families. The **known-gaps** family matters most and is easiest to overlook: a system that writes beautifully from complete data and invents plausibly from incomplete data fails *silently* — every output looks equally confident. Only a fixture that knows what is missing can catch it.

We also run the gates against a **deliberately broken** executor, to prove they fail when they should. A harness that has only ever reported PASS is a harness nobody has tested.

The gate has already caught a real bug. Bare `"interest"` was a blocking term, so the compliant control case *"the client indicated interest in fleet expansion"* was refused. A gate that fires on correct content stops being read within a week. We made the riba senses explicit and downgraded the bare term to a flag.

6. Launch roadmap and integration plan

Phase 1 — Pilot (weeks 1–6)

Deploy in Warba's secure sandbox. Replace synthetic fixtures with the anonymised dataset. Two document types with 3–5 RMs. Run the evaluation harness against real anonymised data and publish the baseline.

Integration surface: the `ContextAssembler` is the single seam. Connecting core banking, CRM, and KYC means implementing source adapters behind it — the generation, validation, screening, and audit layers do not change.

Phase 2 — Shariah and Compliance sign-off (weeks 4–8, parallel)

Shariah Board review of `config/vocabulary.yaml` — deliberately plain YAML for exactly this. Compliance review of the audit trail against CBK record-keeping expectations. Penetration test and a formal data-residency decision, including whether an on-premise model is required (the port abstraction exists for this).

Phase 3 — Credit Memo (weeks 8–12)

Ship DT3 narrative sections **only after** the fabricated-figure gate passes on real data. Bounded to narrative — no ratings, no recommendations, no pricing, enforced by a deterministic guard.

Phase 4 — Scale (weeks 12+)

Full corporate portfolio. Arabic/English bilingual output. Additional document types through configuration. Integration with the bank's document management system.

What has been verified against a live stack, and what has not

We ran this against a real, migrated PostgreSQL database — not just the deterministic test suite — and it caught three bugs that no stub-based test could have found. That is the honest reason live infrastructure matters, and it is worth being precise about exactly what is and is not yet proven.

Verified live (real database, real HTTP requests, no mocks):

- Portfolio scoping — two RMs, provably disjoint client lists via the running API
- The Shariah gate returning HTTP 451 with named rule violations, on a live request
- Fail-closed behaviour — zero documents persisted on a pipeline failure
- **Audit survives the failure it is recording.** The first version of the failure path shared one transaction with the document write, so rolling back the document also rolled back its own audit trail — the exact case FR-039 exists to prevent, and it only showed up under a real rollback. Fixed: the audit write now runs in its own transaction, committed independently.

- The `INSERT, SELECT` -only grant on `audit_event` , and the `is_synthetic` CHECK constraint — both proven by attempting the forbidden operation directly against Postgres, not by testing application code that assumes the privilege exists. Now captured as automated tests (`tests/integration/test_audit_privileges.py`).

Not yet exercised: an actual call to `claude-opus-5` . Every generation guarantee — grounding, gap-marking, fabrication-checking — is verified against a deterministic stub standing in for the model. `pytest tests/evaluation --run-model` is wired and ready; it needs only an `ANTHROPIC_API_KEY` . This is the next action in a pilot, and we would rather find any gap between the two-pass design and real model behaviour there than in a credit memo.

Two smaller items:

- **Baselines are estimates** (SC-001–SC-003) and need validation with real RMs.
 - **English only** in MVP. Bilingual output is scoped, not built.
-

7. Repository guide

Path	What it holds
<code>.specify/memory/constitution.md</code>	Eight non-negotiable principles, v1.0.0
<code>specs/001-ai-client-documentation/spec.md</code>	Full PRD — 44 requirements, 30 NFRs
<code>specs/001-ai-client-documentation/research.md</code>	14 technical decisions with rejected alternatives
<code>specs/001-ai-client-documentation/plan.md</code>	Architecture + 8-gate constitutional check
<code>backend/app/adapters/anthropic_adapter.py</code>	Two-pass generation (the only file importing <code>anthropic</code>)
<code>backend/app/documents/validators.py</code>	Numeric tracing — the anti-hallucination core
<code>backend/config/vocabulary.yaml</code>	The Shariah gate, in reviewable YAML
<code>backend/tests/evaluation/</code>	The accuracy harness and its five gates
<code>README.md</code>	Setup and how to see each guarantee work

8. Why this submission

Most AI documentation tools optimise for how good the output looks. We optimised for whether it can be trusted, and accepted a slower, two-call architecture to get there.

The result is a system that will tell an RM "*I could not find this*" rather than write something plausible — and that refuses to produce a document at all rather than show one containing a figure it cannot support.

For a bank, that is the only version of this product worth deploying.